

TRAVAILLER ENSEMBLE À DISTANCE



Collaborer en temps réel ou en différé :
quels outils pour quels usages ?

Claudia-Lavinia Ignat, Gérald Oster,
Quentin Acher, Victor de Moura Netto, Ludovic Paillat
Équipe COAST,
LORIA/Inria Centre de l'Université de Lorraine

Résumé

- ▶ Cet atelier propose une mise en situation autour de deux types de systèmes collaboratifs : les éditeurs synchrones en temps réel (type pad collaboratif) et les outils asynchrones de gestion de versions (type dépôt partagé, suivi de modifications).
- ▶ Les participants expérimenteront ces outils à travers une activité de corédaction, afin d'en identifier les usages possibles, les avantages, les limites et les conditions de réussite selon les contextes de travail.

Travailler ensemble en temps-réel (collaboration synchrone)



30 minutes

Édition collaborative temps-réel

Objectifs pédagogiques :

- Expérimenter l'écriture collaborative synchrone
- Ressentir les problèmes de coordination
- Identifier des stratégies simples pour mieux travailler ensemble

Outils phares :

- Google Docs, Etherpad, CodiMD, Framapad, CryptPad, Docs, Overleaf/ShareLaTeX

Avantages :

- Tout le monde écrit dans le même document en même temps
- Co-construction de documents sans échange de versions
- Réduction des délais de production
- Suivi des contributions (historique, commentaires)
- Risques : confusion, écrasement, perte de sens sans structure

Démonstration – 5 minutes

Déroulement :

- Tous des éditeurs collaboratifs multi-plateforme (en ligne) en temps réel au format Markdown (MD)

Outils :

- Docs - <https://docs.numerique.gouv.fr/> 
- CodiMD - <https://codimd.apps.education.fr/>
- Et bien d'autres plateformes

Liens partagés :

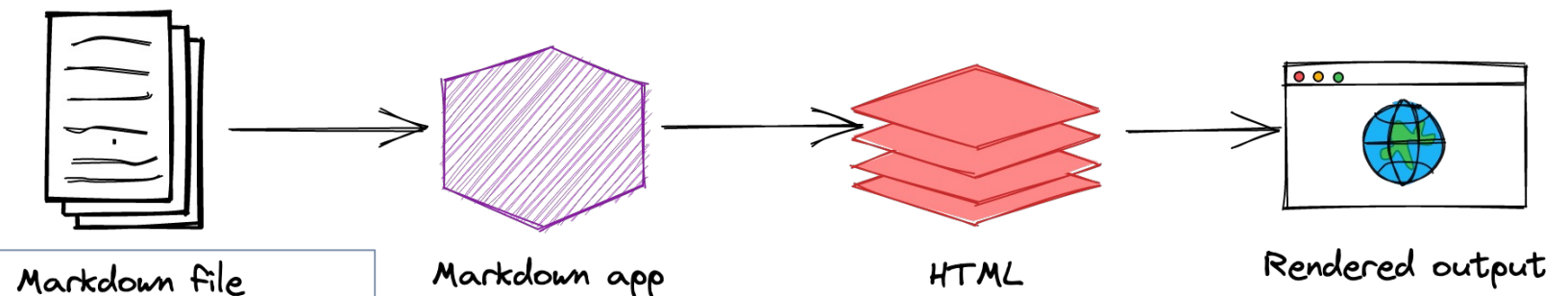
- <https://notes.inria.fr/cC17r2BmQUCLa5Hv2-G36g>
ou <https://tinyurl.com/4pjjkxd6>
- <https://docs.numerique.gouv.fr/docs/0e489d8b-f8f1-4e43-a334-c61eeeb1a02e/>
ou <https://tinyurl.com/t3na2k59>

Ces documents peuvent ne plus être
accessibles après l'atelier

Ces documents peuvent ne plus être
accessibles après l'atelier

Qu'est-ce que Markdown ?

- Langage de balisage léger, simple à apprendre, utilisé pour formater du texte.
- Format texte « brute » indépendant du format de rendu final (HTML, PDF, etc.)



```
# Titre niveau 1
## Titre niveau 2
```

```
**Texte en gras**
*Texte en italique*
- Liste à puces
1. Liste numérotée
```

```
[Un lien](https://exemple.com)
```

Source : <https://www.markdownguide.org/getting-started/>

<https://codimd.apps.education.fr/>

<https://codimd.apps.education.fr/s/H6nqsVq2y>



Atelier 1 – Rédaction – 15 minutes

Groupe :

- 4 à 5 personnes par groupe

Mise en œuvre :

- Groupe 1 : <https://notes.inria.fr/TK0BedMrQO2d17nwWL5G7A>
- Groupe 2 : <https://notes.inria.fr/-CWZ7N4uSM2IZ37tVK4qAQ>
- Groupe 3 : https://notes.inria.fr/ajN1n_x0Rum6zY-7CwTsTg
- Groupe 4 : <https://notes.inria.fr/iZxhf994SAuYYva6jAOL4A>
- Groupe 5 : https://notes.inria.fr/MHx7tAi6TAqAFPEbiX_zKQ

Ces documents peuvent ne plus être
accessibles après l'atelier

Consigne :

À partir du thème « Les usages du numérique dans la formation des enseignants », rédigez ensemble une courte note (environ 15 lignes) destinée à un recteur. Tout le monde écrit en même temps. Pas de concertation préalable.

Observations libres

Atelier 1 – Débriefing – 10 minutes

Débriefing :

- Qu'avez-vous apprécié dans ce mode de travail ?
- Qu'avez-vous ressenti en écrivant tous ensemble ?
- Avez-vous perdu du contenu ? Avez-vous été en conflit avec quelqu'un ?
- Que pourrait-on faire pour que ça se passe mieux
- Quelles limites ou freins voyez-vous à l'adoption dans vos pratiques ?

Atelier 1 – Débriefing – 10 minutes

À retenir :

- Toujours prévoir une phase de structuration (plan, rôles, zones)
 - Définir un plan commun
 - Se répartir les sections ou les rôles
 - Travailler d'abord sur le fond
 - Puis, relire ensemble
- Utiliser les outils intégrés
 - Commentaires, chat, historique
- La coordination est la clé d'une collaboration efficace, surtout en synchrone

Gestion de versions (distribuée) (collaboration asynchrone)



50 minutes

Gestion de versions

Objectifs pédagogiques :

- Comprendre les enjeux et bénéfices de la gestion de versions.
- Apprendre à utiliser Git pour le suivi d'un projet.
- Savoir configurer Git et effectuer les premières commandes.
- Collaborer à plusieurs sur un projet commun via Git et GitLab.

Outils phares :

- CVS, Git, GitLab, GitHub

Avantages :

- Historique clair des modifications.
- Revenir en arrière à tout moment.
- Travailler à plusieurs sans écraser le travail des autres.
- Documentation implicite de l'évolution du projet.

Introduction à la gestion de versions

Motivations :

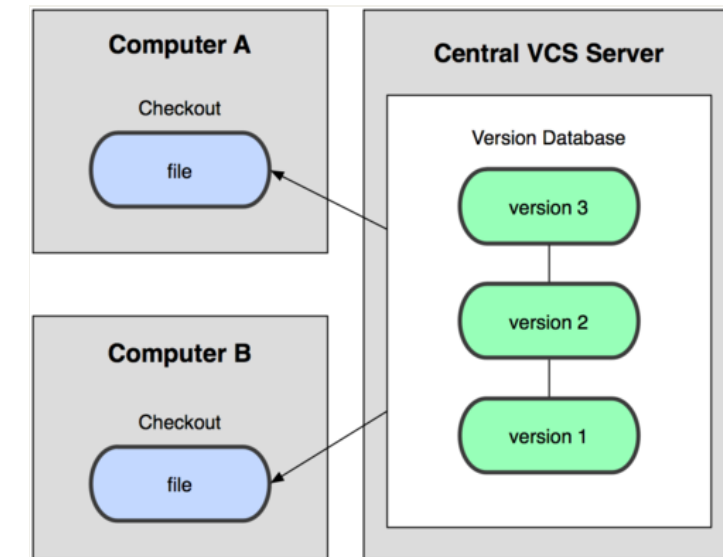
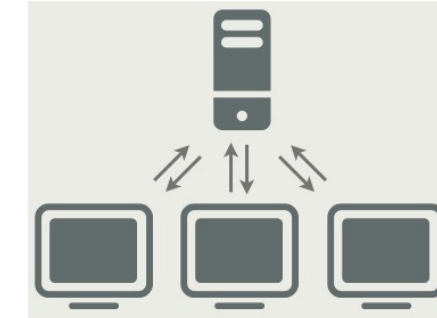
- Gérer les données en conservant systématiquement la version précédente
- Archivage
- Récupération d'une ancienne version
- Comparaison de la version courante par rapport à une ancienne version

Utilités des versions :

- Versions historiques : repartir sur une version antérieure en cas d'erreur (*short/long term undo*) – ex : version du 19/01/2023, version #227, etc.
- Versions alternatives (*branching*) : différents stades d'implémentations (stable, corrections du bug #22, ajout de d'une fonctionnalité, différentes conceptions, etc.)

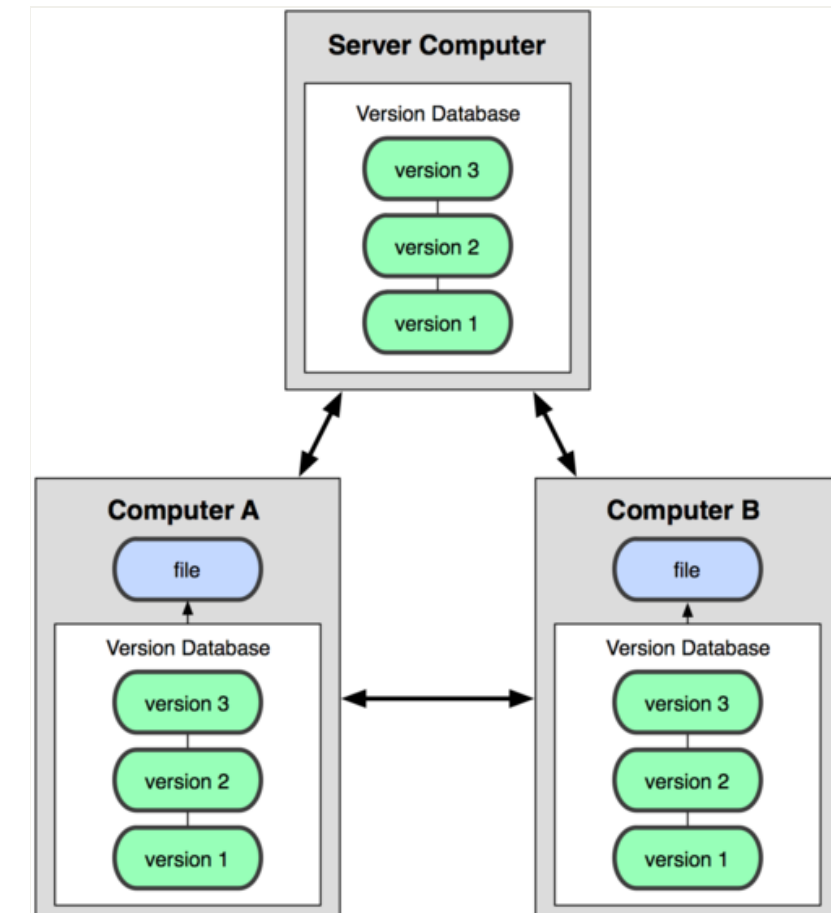
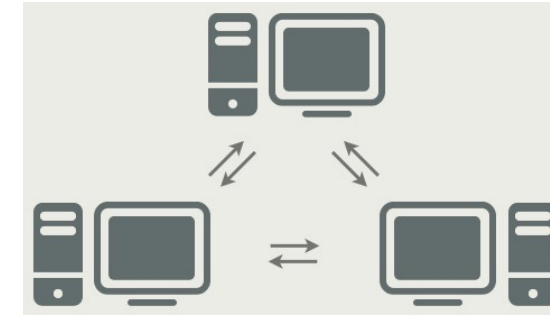
Gestion de versions – centralisée

- Le serveur détient tout l'historique du projet
- Les utilisateurs possèdent une copie des fichiers correspondant à leur état actuel du projet
- La grande majorité des opérations sont réalisées en communiquant avec le serveur
- Le partage de code passe obligatoirement par le serveur
- Exemples : Subversion, CVS



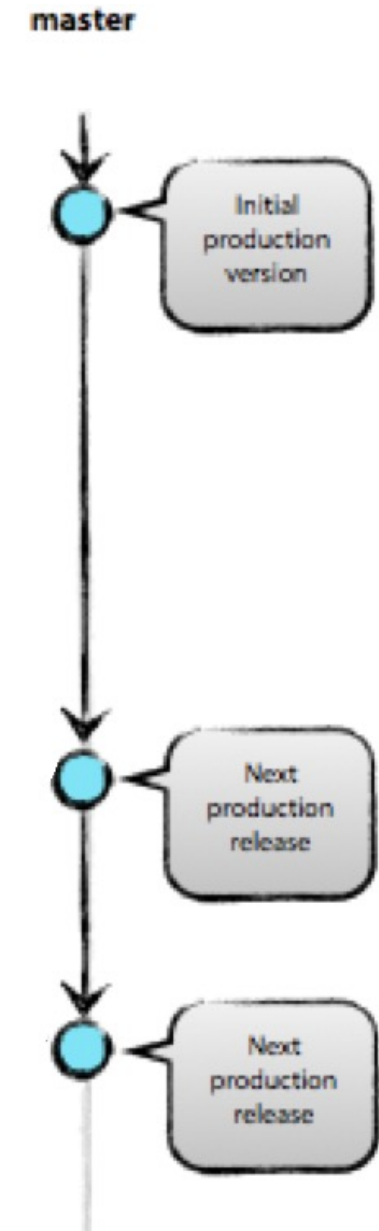
Gestion de versions – décentralisée

- Chaque utilisateur possède un dépôt local complet, contenant tout l'historique du projet
- Les utilisateurs possèdent une copie des fichiers correspondant à leur état actuel du projet
- Les opérations sont réalisées localement (hors ligne)
- Le partage de code peut se faire entre utilisateur directement (ou passer par un serveur)
- Exemples : Git, Mercurial



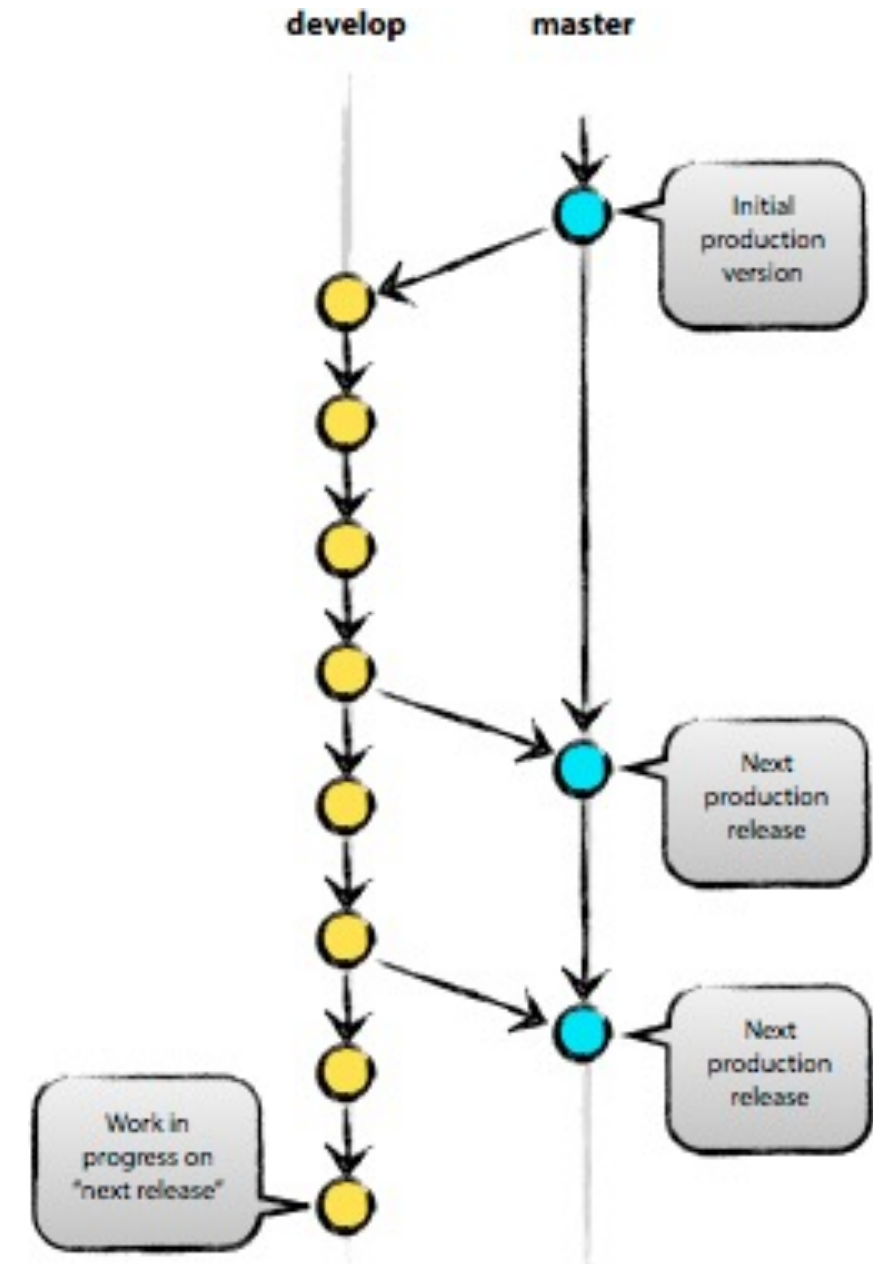
Gestion de versions - vocabulaire

- Révision : séquence de versions
- Branches : alternatives au développement master, develop, feature
- Fusion de branches : fusion des développements parallèles
- Tags : Identification symbolique de révision



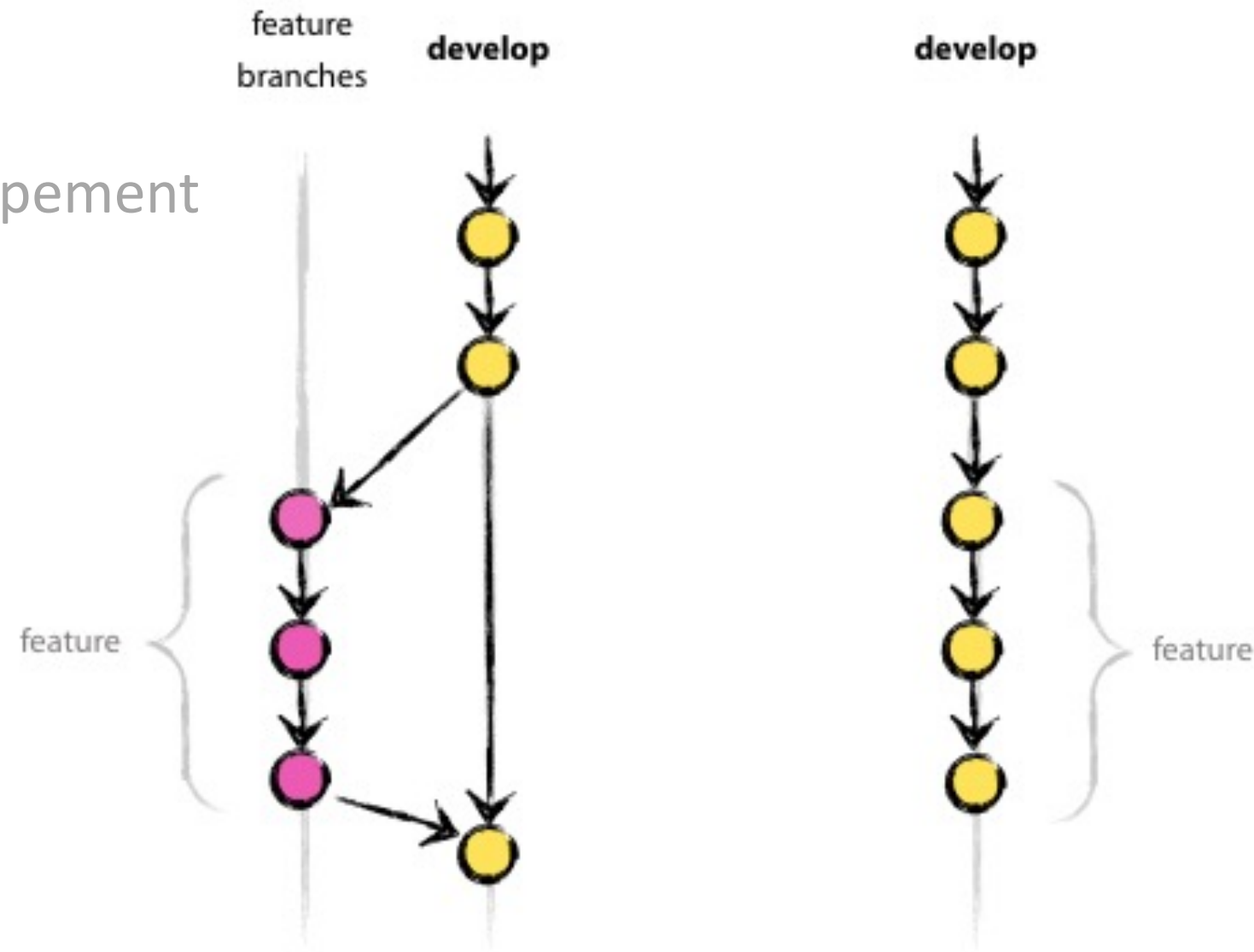
Gestion de versions - vocabulaire

- Révision : séquence de versions
- Branches : alternatives au développement master, develop, feature
- Fusion de branches : fusion des développements parallèles
- Tags : Identification symbolique de révision



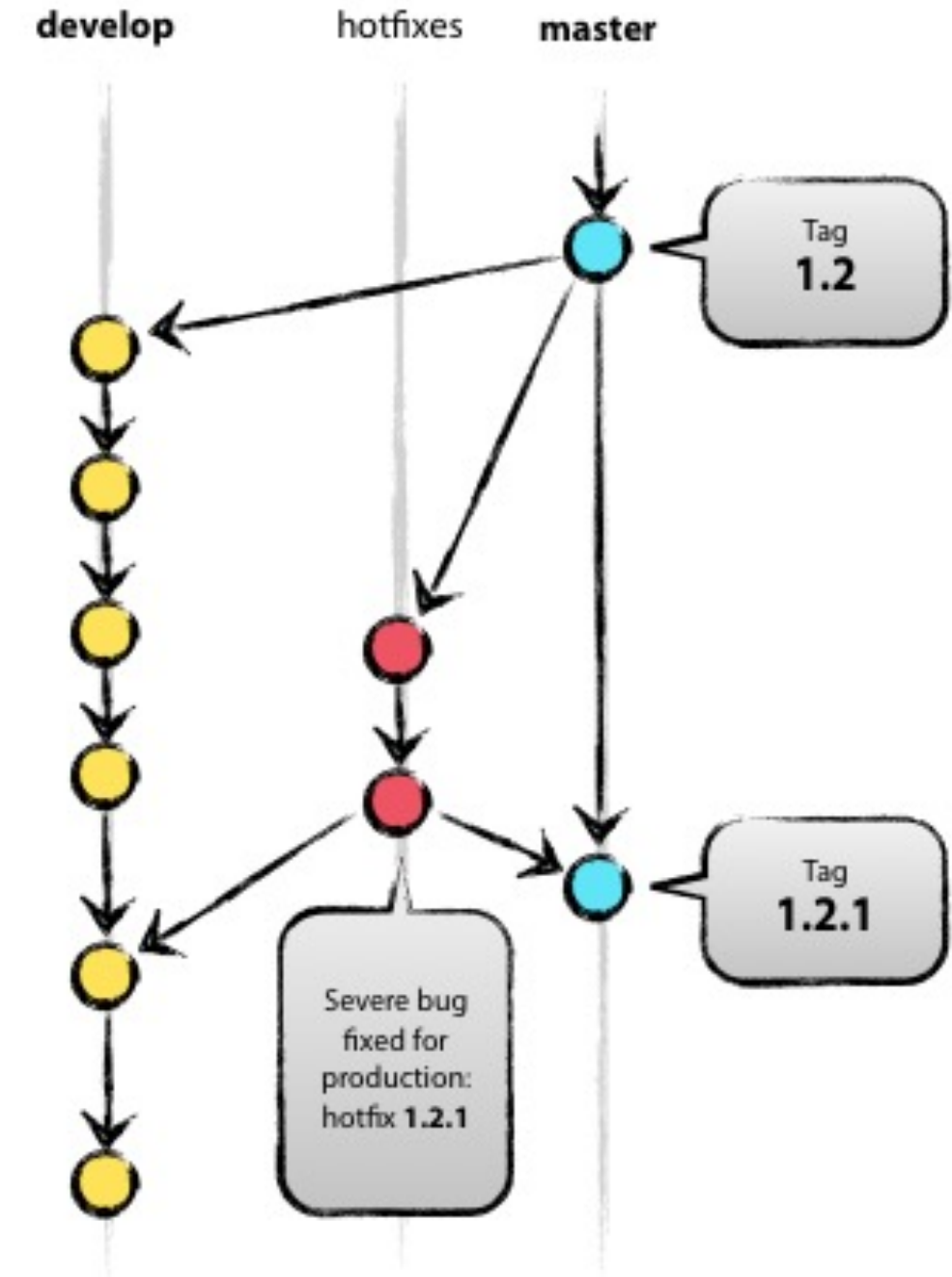
Gestion de versions - vocabulaire

- Révision : séquence de versions
- Branches : alternatives au développement master, develop, feature
- Fusion de branches : fusion des développements parallèles
- Tags : Identification symbolique de révision



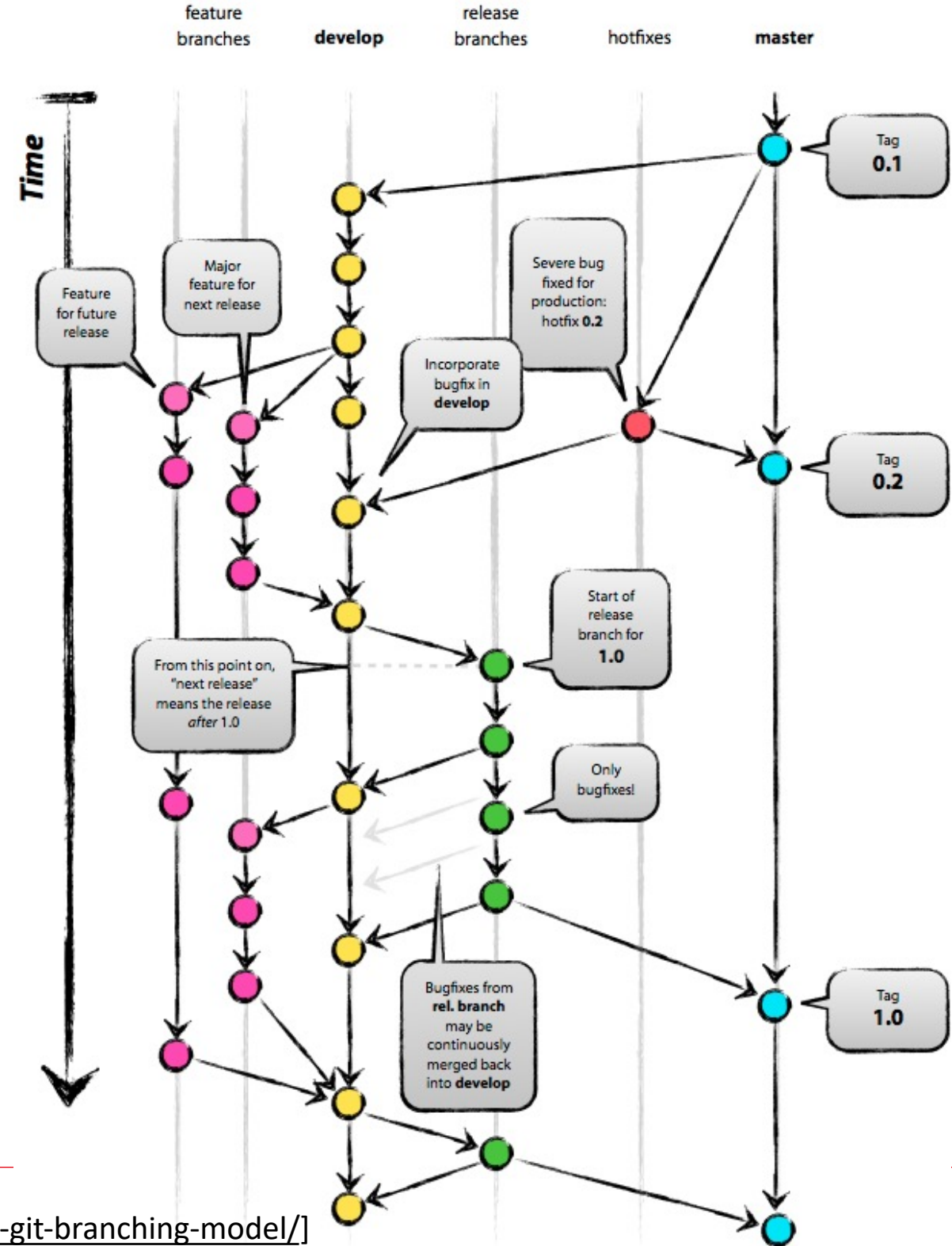
Gestion de versions - vocabulaire

- Révision : séquence de versions
- Branches : alternatives au développement master, develop, feature
- Fusion de branches : fusion des développements parallèles
- Tags : Identification symbolique de révision



Gestion de versions - vocabulaire

- Flux de travail complets



Premiers pas avec Git & GitLab



<https://git-scm.com/>

Qu'est-ce que git ?

- Git est un gestionnaire de versions décentralisé (DVCS)
- Créé en 2005 par Linus Torvald pour le développement du noyau Linux
- Un des DVCS les plus populaires (estimée à 90% part de marché)
- Multi-plateformes

Qu'est-ce que GitLab ?

- GitLab est une forge logicielle
- Hébergement de projets dont des dépôts git
- Gestion de tickets
- Wiki et documentation de projet
- CI/CD intégré (Intégration et déploiement continu)



<https://gitlab.com/>

<https://forge.apps.education.fr>

Premiers pas avec Git



Installation

- Linux / macOS : utilisez votre gestionnaire de paquets

```
$ sudo apt install git-all
```

```
$ brew install git
```
- Windows : **Git for Windows** - <https://git-scm.com/downloads/win>

Vérification & Configuration (dans un terminal)

```
$ git -version
```

```
$ git config --global user.name "Gerald Oster"
```

```
$ git config --global user.email "gerald.oster@loria.fr"
```

Premiers pas avec Git & GitLab

Accéder à un projet sur la forge

<https://gibson.telecomnancy.univ-lorraine.fr/pnf/demo>

Duplication d'un dépôt

```
$ git clone https://gibson.telecomnancy.univ-lorraine.fr/pnf/demo.git
```

Modifiez le fichier README.md avec votre éditeur de texte habituel

```
$ git status
```

```
$ git add README.md
```

```
$ git commit -m "premier changement"
```

```
$ git log
```

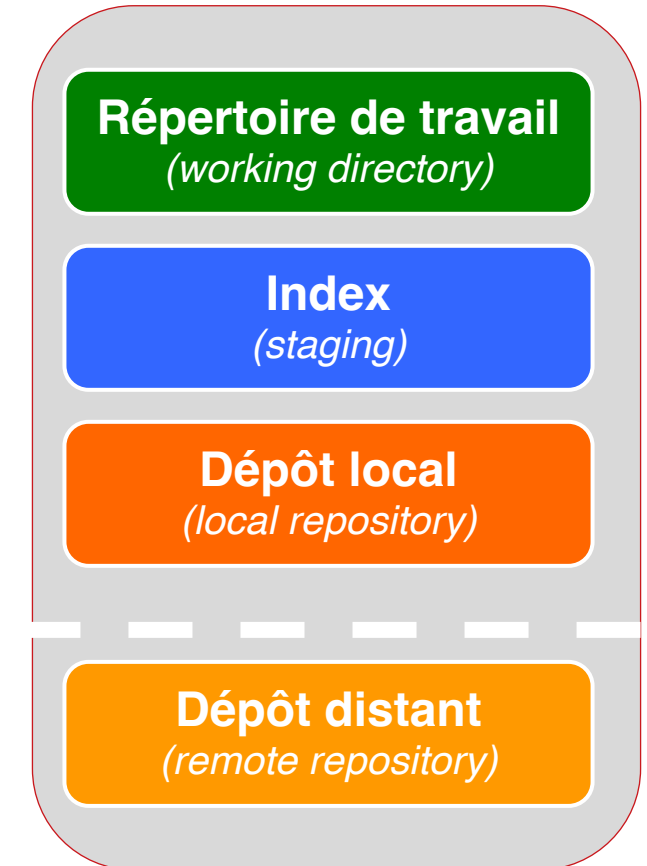


Premiers pas avec Git – Principe de fonctionnement d'un dépôt

- Git conserve un historique des instantanées (*snapshots*) d'un projet
- Création d'un dépôt (*repository*) vide / ou clonage d'un dépôt existant
- Alimentation du dépôt par l'intermédiaire de *commits*
- Un commit contient :
 - un ensemble de modifications de données, suite aux manipulations des fichiers du projet (création, édition, suppression, renommage, etc.)
un journal associé : commentaire sur la nature des modifications
 - des méta-informations : identifiant de commit, auteur, date

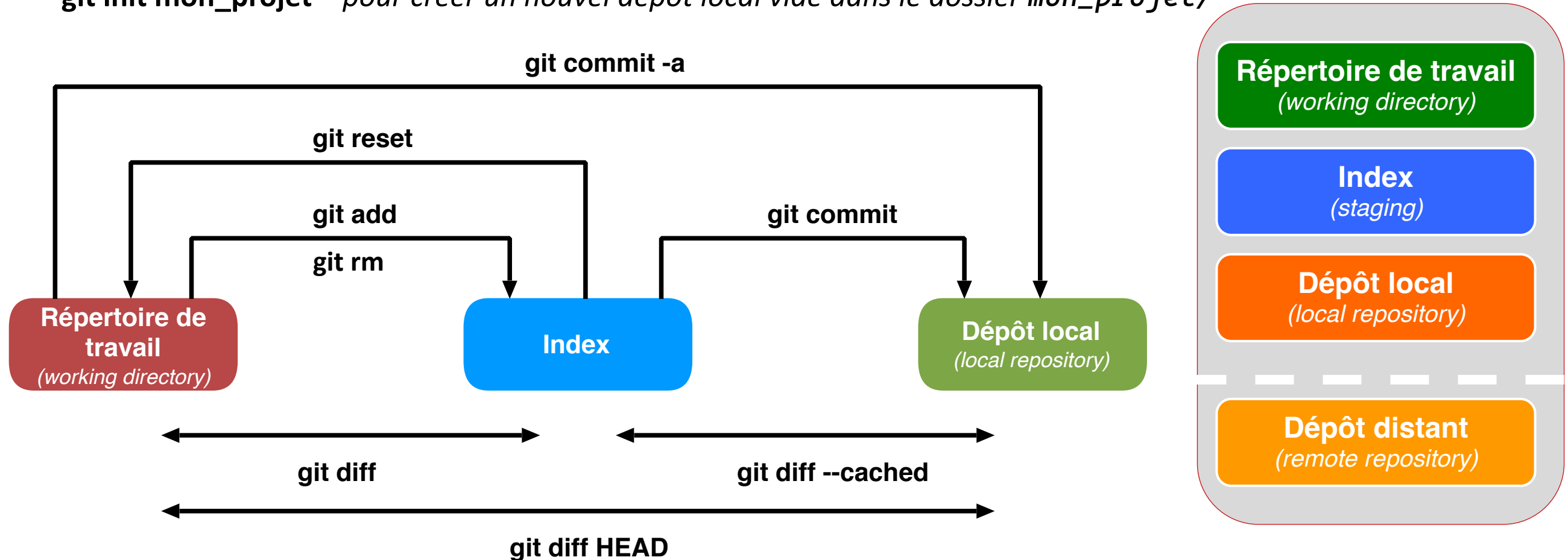
Premiers pas avec Git – Différents niveaux de stockage

- Répertoire de travail qui contient :
 - la copie locale des fichiers du projet
 - à sa racine un répertoire .git (configuration et données du projet)
- Un index :
 - espace temporaire utilisé pour préparer le transfert des changements du répertoire de travail vers le dépôt local
 - permet de choisir les sous ensembles des changements à valider (*commit*)
- Un dépôt local :
 - Contient la totalité des versions du projet et les métadonnées
 - Propre à chaque utilisateur
- Un dépôt distant (optionnel) :
 - Intrinsèquement similaire à un dépôt local, peut ne pas contenir d'espace de travail



Premiers pas avec Git – Principales commandes locales

`git init mon_projet` – pour créer un nouvel dépôt local vide dans le dossier `mon_projet/`





Premiers pas avec Git – Principales commandes « distantes »

- Clone : Pour obtenir le code d'un projet et son histoire à partir de n'importe quel dépôt, il suffit de le dupliquer

```
$ git clone https://gibson.telecomnancy.univ-lorraine.fr/pnf/demo.git
```

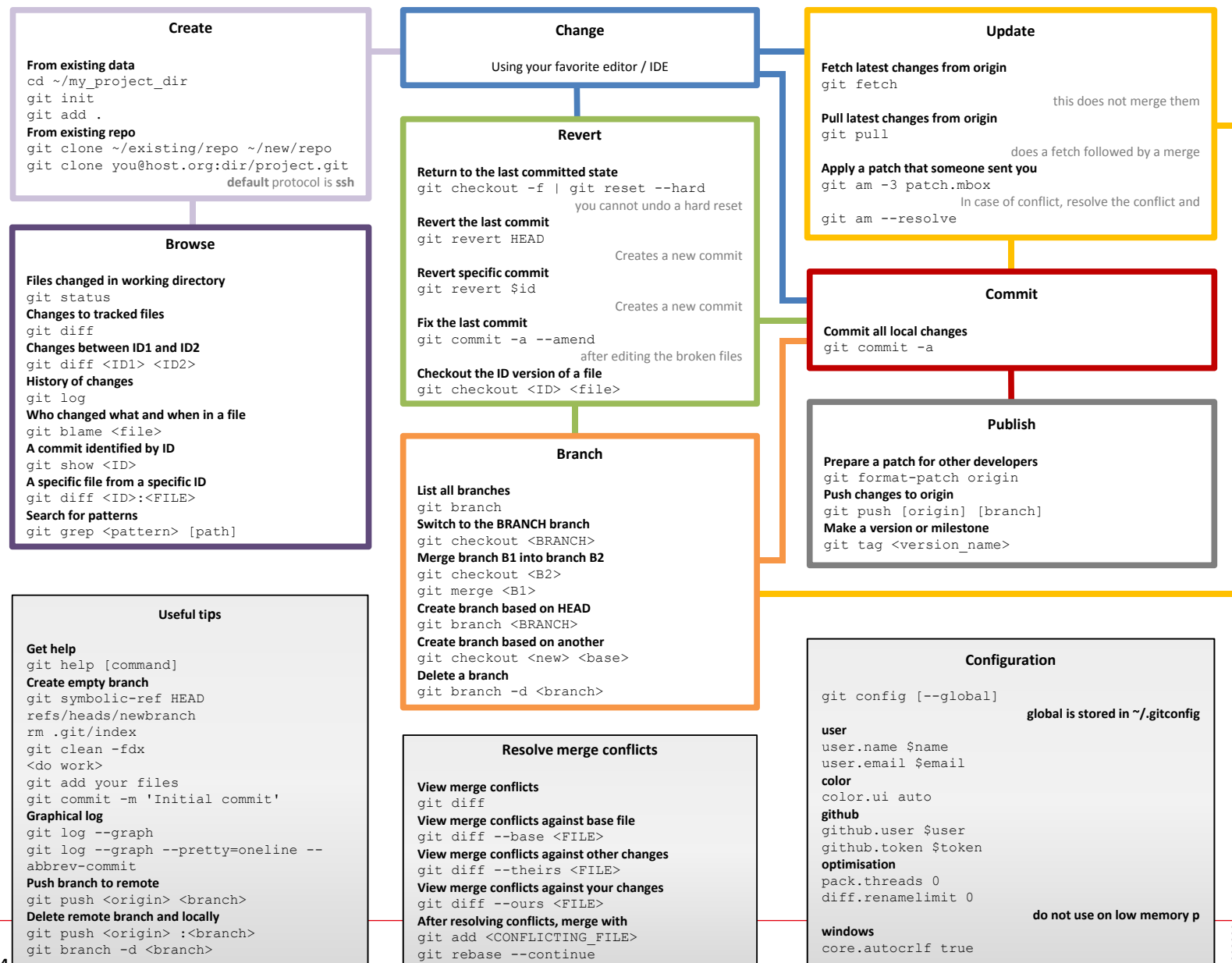
- Push : Pour publier des modifications validées (commits) vers un dépôt distant

```
$ git push [origin] [main]
```

- Pull : Pour obtenir les derniers changements publiés sur un dépôt distant

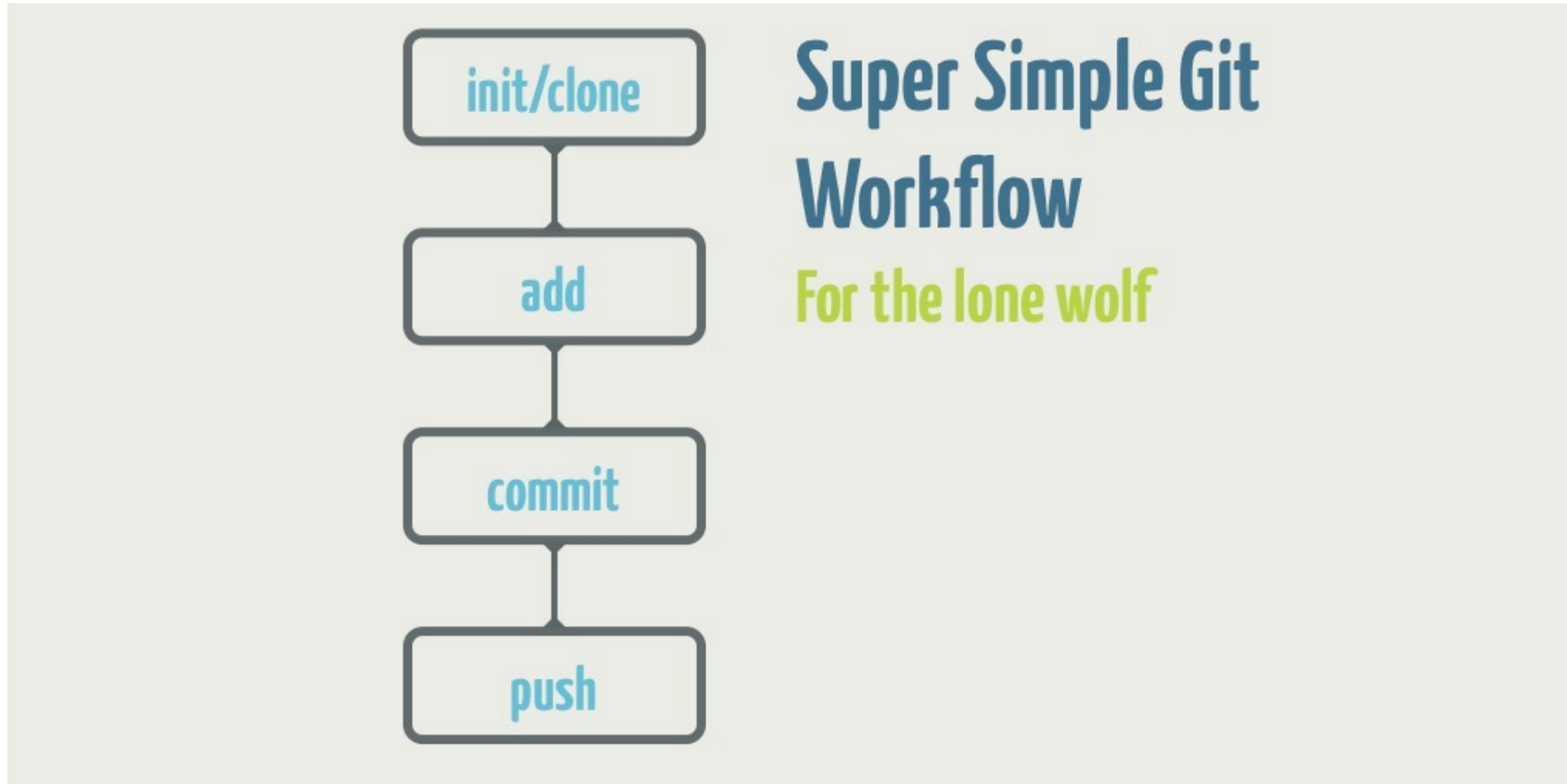
```
$ git pull [origin] [main]
```

Premiers pas avec Git – Quelques commandes...

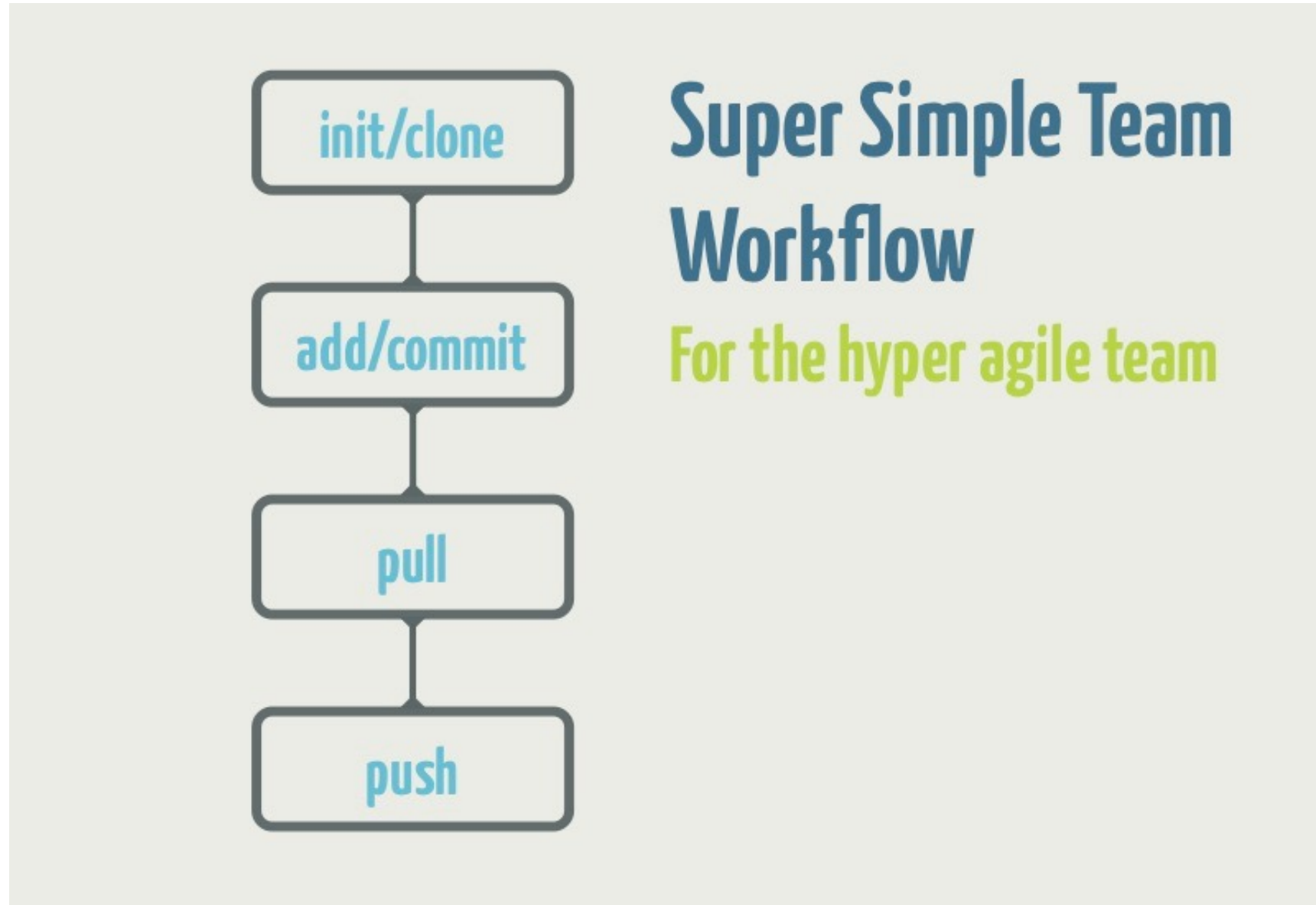


\$ git help
\$ git help --all

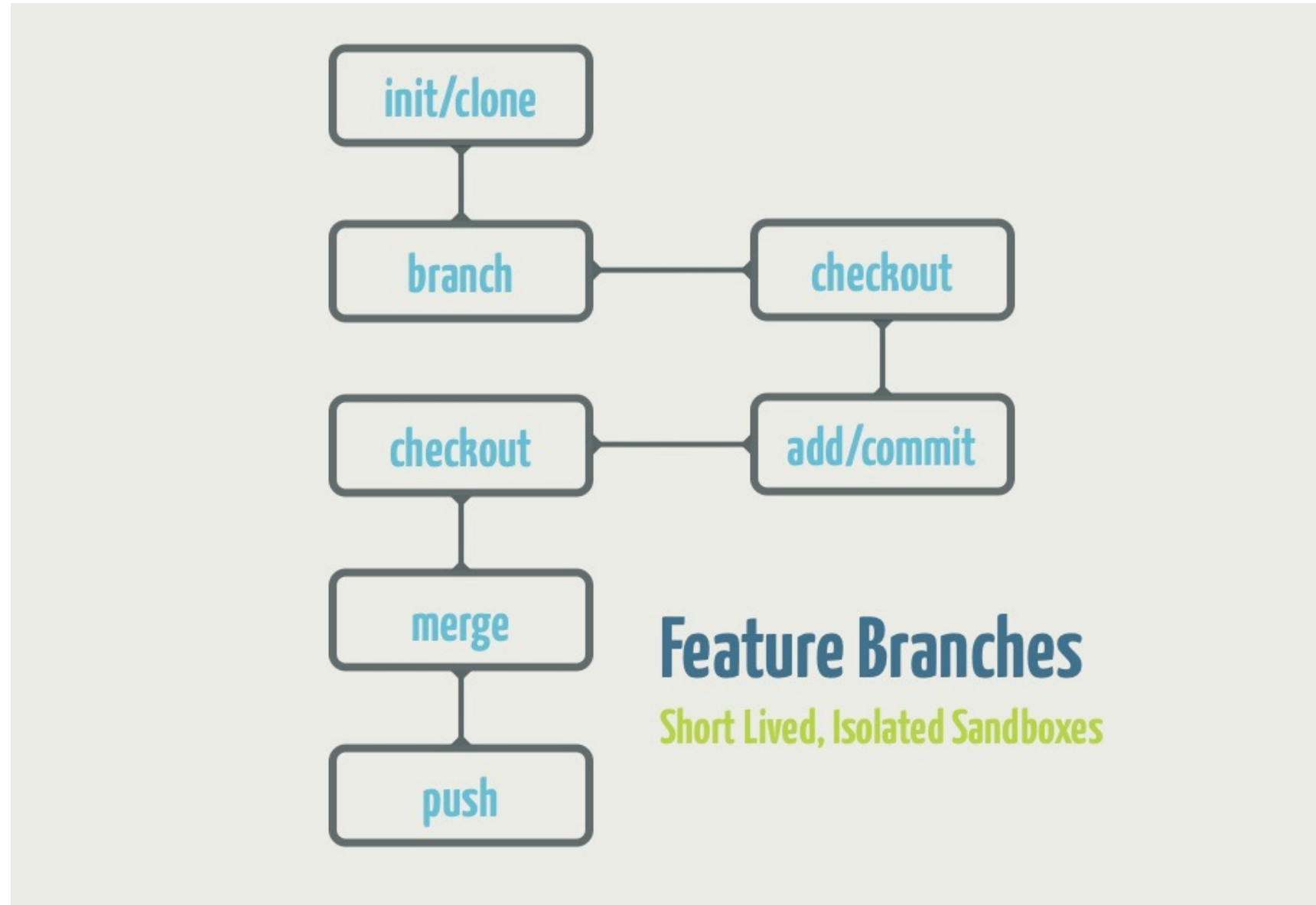
Premiers pas avec Git – Flux de travail



Premiers pas avec Git – Flux de travail

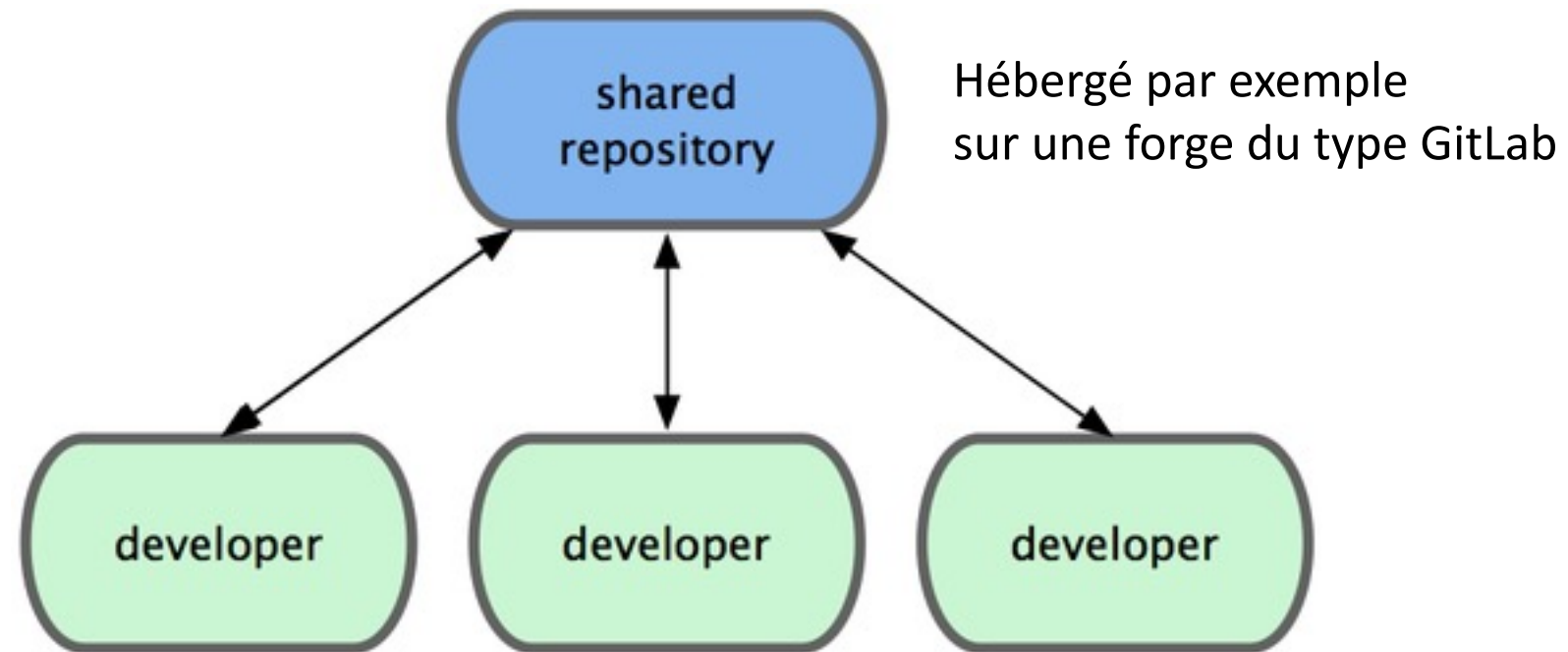


Premiers pas avec Git – Flux de travail



Premiers pas avec Git – Organisation de la collaboration

- Sur le modèle « centralisé »

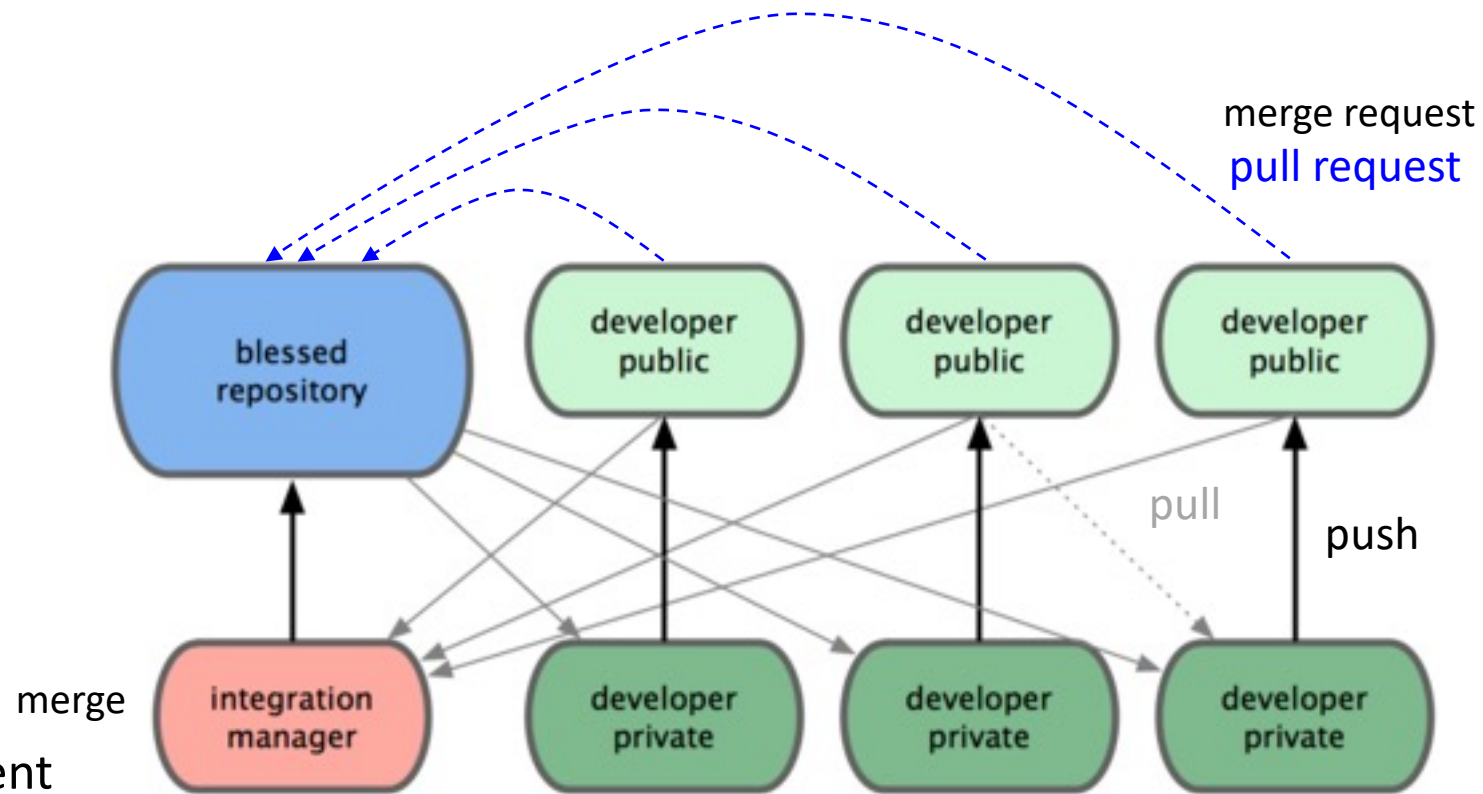


chaque développeur possède son dépôt local

Premiers pas avec Git – Organisation de la collaboration

- Intégrateur / Manageur

chaque développeur possède son dépôt « public »
et effectue des demandes d'intégration sur le « dépôt de référence »

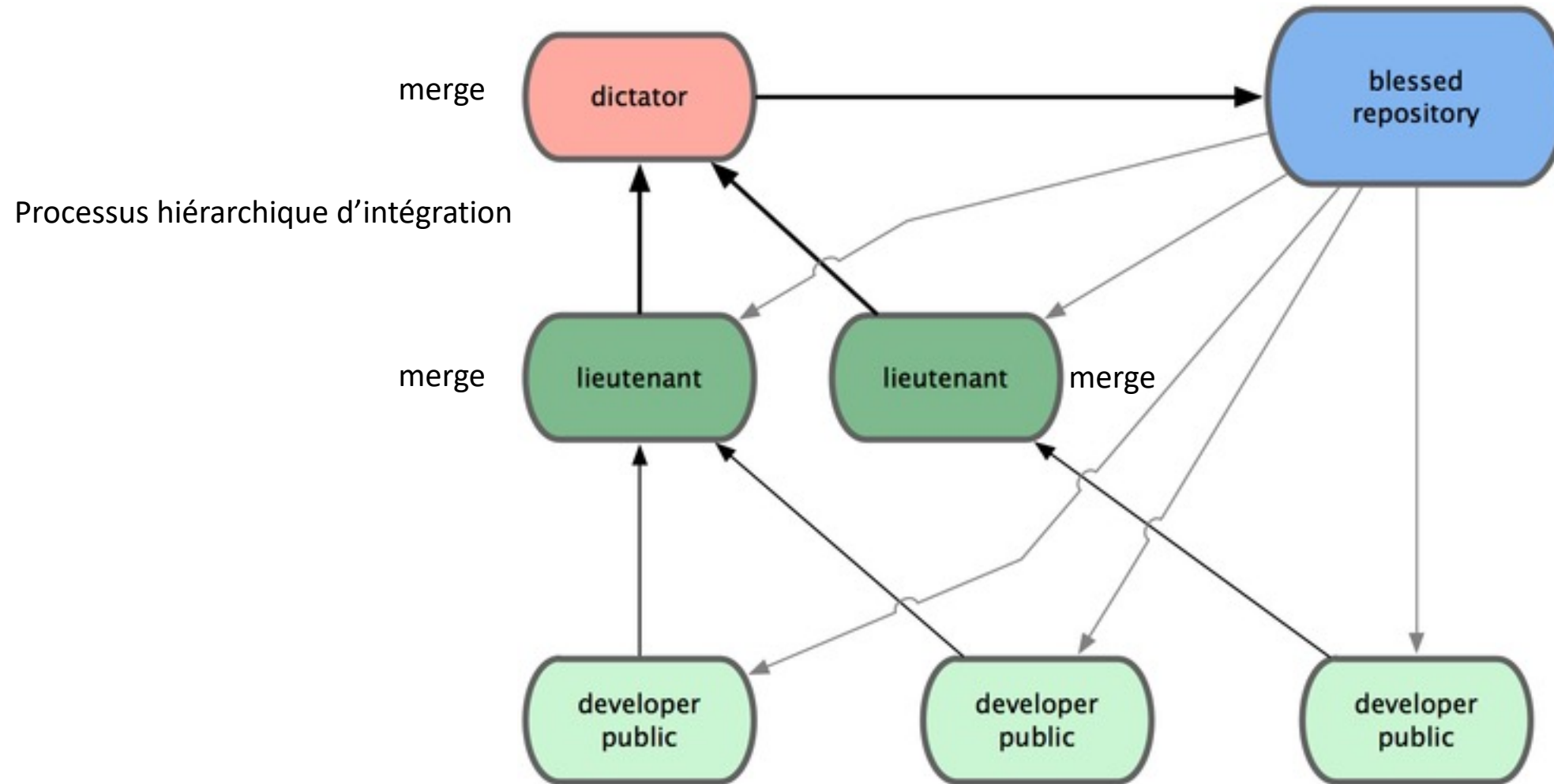


Les intégrateurs choisissent
accepter ou non les demandes

chaque développeur possède son dépôt local

Premiers pas avec Git – Organisation de la collaboration

- Dictateur / Lieutenants (modèle utilisé pour la gestion du noyau Linux)



Premiers pas avec Git – *Best Practices*

- Commit related changes
- Commit often
- Don't commit half-done work
- Test code before commit
- Write good commit messages
- Use branches
- Pull often



Atelier 2 - Développement collaboratif en Python

Groupe :

- 4 à 5 personnes par groupe

Mise en œuvre :

- 1 personne bifurque (*fork*) le dépôt du projet
<https://gibson.telecomnancy.univ-lorraine.fr/pnf/dev>
- Toutes les personnes clonent en local le dépôt du nouveau projet
- Répartissez vous les rôles (création du code, écriture de la documentation, écriture des tests unitaires)
- Collaborez pour obtenir rapidement un projet fonctionnel, documenté et dont la correction est testée

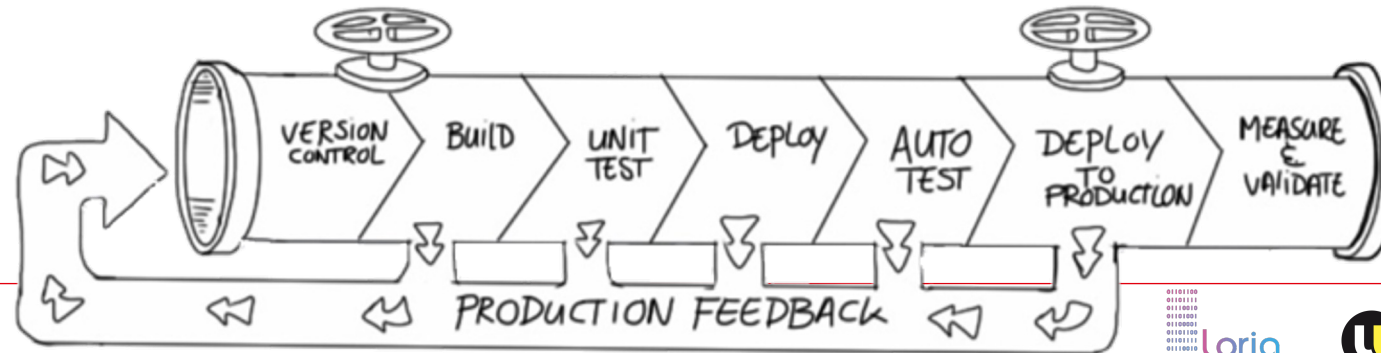
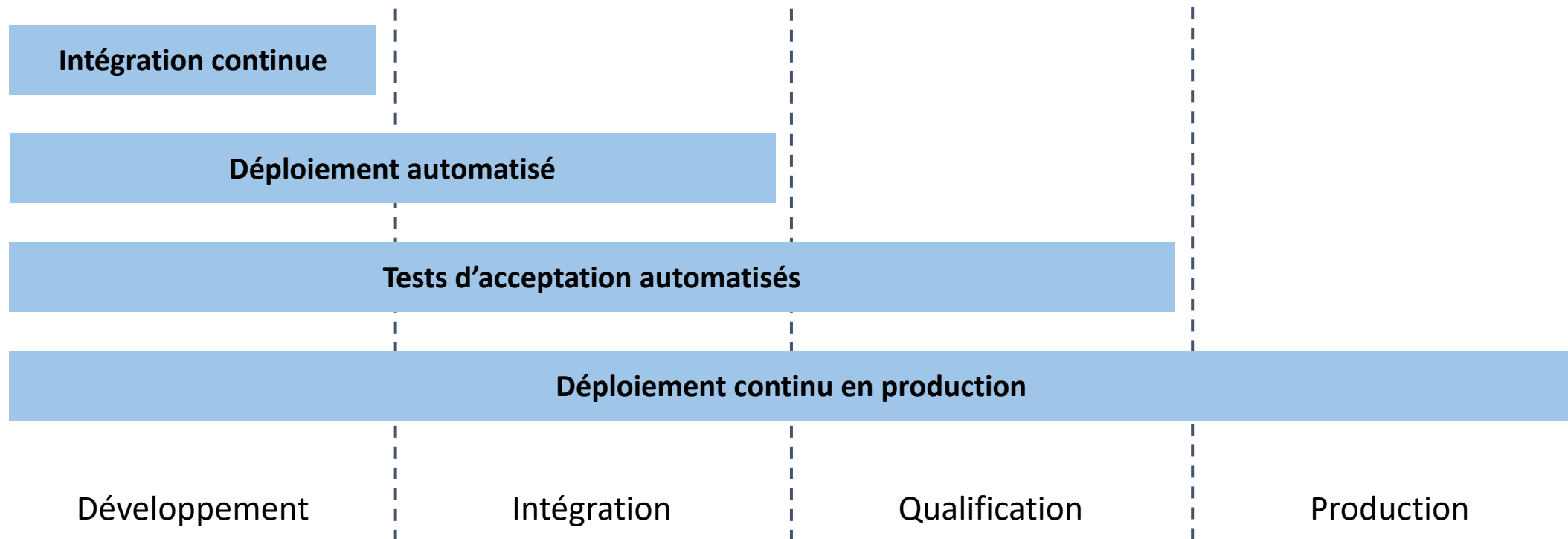
Ces documents peuvent ne plus être
accessibles après l'atelier



CI / CD – Intégration et déploiement continus

- Une forge permet d'automatiser des séquences de tâches (*pipeline*) régulièrement (fréquence, chaque commit, etc.)
- Les tâches peuvent être diverses :
 - contrôle de la qualité/conformité,
 - compilation,
 - exécution de tests (unitaires, integration, acceptation)
 - génération d'artefacts (distributions, page/site web, documentations, etc.)
 - déploiement (bleu/vert), mise en production, test A/B

CI / CD – Intégration et déploiement continus





Atelier 2b - Rédaction collaborative en Markdown + déploiement

Groupe :

- 1 personnes ou plusieurs par groupe

Mise en œuvre :

- Créez un nouveau projet à partir d'un modèle préexistant (dans la liste de ceux proposé par GitLab)
- Explorez les possibilités
- Consulter le fichier `.gitlab-ci.yml`

 **Thank you! Questions ?**